

HumanMaximizer AI Lead Generation

Architecture & Design Document

Razor Infotech Pvt Ltd — AI Architect / GenAI Engineer Assignment

Open Source LLM Strategy

HumanMaximizer AI Lead Generation System

Razor Infotech Pvt Ltd — AI Architect / GenAI Engineer Assignment

Overview

This document explains the open-source LLM selection, quantization strategy, and GPU requirements for the HumanMaximizer AI Lead Generation system. All inference runs locally via **Ollama** — no external API calls, no per-token costs, full data privacy.

1. Model Selection

Candidates Evaluated

Model	Parameters	Context	License	Strengths	Weaknesses
Mistral-7B-Instruct-v0.3	7B	8k	Apache 2.0	Strong instruction-following, Indian-English well-represented, structured JSON output	Slightly behind Llama-3 on open benchmarks
Llama-3-8B-Instruct	8B	8k	Meta Llama 3	Excellent general quality, strong reasoning	Larger memory footprint, Meta license restrictions for commercial use
Phi-3-Mini-4k-Instruct	3.8B	4k	MIT	Very fast, small footprint	4k context too short for multi-paragraph research summaries + RAG chunks

Model	Parameters	Context	License	Strengths	Weaknesses
Gemma-7B-Instruct	7B	8k	Gemma ToU	Strong quality, Google-backed	Restrictive usage policy (no commercial redistribution), less instruction-tuned for B2B sales copy

Decision: Mistral-7B-Instruct-v0.3

Selected for 5 reasons:

1. Instruction-following quality

Mistral-7B-Instruct-v0.3 consistently produces structured output in our required formats — SUBJECT: /BODY: email splits, numbered reasoning, JSON-compatible responses — without hallucinating structural elements.

2. Indian-English domain fit

The model has strong representation of Indian business English in its training corpus — company names, compliance terms (ESIC, PF, CLRA, PT), city names (Pune, Bangalore, Mumbai), and B2B sales communication patterns.

3. 8k context window

The pipeline injects up to 5 RAG chunks (~2,500 chars) + company profile (~800 chars) + prompt template (~400 chars) into a single LLM call. Phi-3's 4k window is insufficient. Mistral's 8k handles all three agents comfortably.

4. Apache 2.0 license

Fully open for commercial use without usage restrictions. Llama-3 requires Meta license agreement. Gemma has Google's Terms of Use. Mistral is the cleanest license for a production system.

5. Single model for all 3 agents

One model deployment serves ResearchAgent (summary), QualificationAgent (reasoning), and SalesAgent (email + LinkedIn). This simplifies Ollama configuration and reduces VRAM requirements compared to running separate specialized models.

Model Benchmark Comparison

Task	Mistral-7B	Llama-3-8B	Phi-3-Mini	Gemma-7B
Structured output (JSON/format)	■ Excellent	■ Excellent	■■ Good	■ Good
Long context (8k)	■ Yes	■ Yes	■ 4k only	■ Yes
Indian-English	■ Good	■ Good	■■ Limited	■■ Limited
Commercial license	■ Apache 2.0	■■ Meta License	■ MIT	■■ Gemma ToU
VRAM at Q4_K_M	■ 4.4 GB	■■ 5.0 GB	■ 2.3 GB	■ 4.4 GB

Task	Mistral-7B	Llama-3-8B	Phi-3-Mini	Gemma-7B
Overall	Selected	Runner-up	Too small	License concern

2. Quantization Strategy

What is Quantization

Quantization reduces model weight precision from 32-bit or 16-bit floating point to lower-bit integer representations. This reduces:

- **VRAM usage** (critical for local deployment)
- **Inference latency** (integer math is faster than float math on most hardware)
- **Model file size** (7B FP16 = ~14 GB → 7B Q4_K_M = ~4.4 GB)

Format: GGUF Q4_K_M via llama.cpp / Ollama

GGUF (GPT-Generated Unified Format) is the standard format for quantized LLMs in Ollama and llama.cpp.

Q4_K_M decoded:

Part	Meaning
Q4	4-bit integer weights (down from 16-bit float) — 75% memory reduction
K	K-quants method — mixed precision per tensor block; attention heads retain more bits than feed-forward layers
M	Medium variant — balanced between Q4_K_S (smaller, lower quality) and Q4_K_L (larger, higher quality)

Why Q4_K_M Specifically

Quantization	VRAM (7B)	Quality vs FP16	Notes
FP16 (no quant)	~14 GB	100%	Baseline — requires A100/H100
Q8_0	~7.7 GB	~99%	Near-perfect quality, too large for 8 GB GPU
Q4_K_M	~4.4 GB	~97%	Selected — fits 8 GB GPU with KV cache headroom
Q4_K_S	~4.1 GB	~96%	Slightly smaller, slightly worse on structured output
Q3_K_M	~3.3 GB	~93%	Measurable JSON output corruption observed in structured tasks

Quantization	VRAM (7B)	Quality vs FP16	Notes
Q2_K	~2.7 GB	~85%	Unacceptable quality degradation for production use

Key insight: Q4_K_M delivers **97% of FP16 quality at 31% of the VRAM cost**. Q3 and below show measurable degradation in structured output tasks (malformed JSON, broken SUBJECT/BODY splits) — making Q4_K_M the minimum viable quantization level for this system.

Embedding Model: nomic-embed-text

The RAG pipeline uses a separate embedding model — **nomic-embed-text** — for ChromaDB vector indexing and retrieval:

Property	Value
Parameters	137M
Embedding dimension	768-dim
Precision	FP16
VRAM	~270 MB
Context	8192 tokens

nomic-embed-text runs concurrently with Mistral-7B within the same Ollama instance. Its negligible VRAM footprint means it does not compete with the LLM for memory.

3. GPU Requirements

Development (Current Setup)

OLLAMA_NUM_GPU=0 → CPU-only mode
Inference speed: ~5–15 tokens/second
Pipeline time: 60–120 seconds per lead
Use case: Development, testing, demos

Recommended GPU Configurations

Target	GPU	VRAM	Tokens/sec	Pipeline Time	Notes
Development	RTX 3070 / RTX 4060 Ti	8 GB	~40–60	~15–25 sec	Q4_K_M + nomic-embed-text fit simultaneously
Production	RTX 4090	24 GB	~80–120	~8–12 sec	Run Q8_0 for better quality; higher concurrency
Cloud (budget)	T4 (Google Colab / GCP)	16 GB	~50–70	~12–18 sec	Handles Q4_K_M comfortably

Target	GPU	VRAM	Tokens/sec	Pipeline Time	Notes
Cloud (production)	A10G (AWS g5.xlarge)	24 GB	~80–100	~8–15 sec	Recommended for production workloads
CPU fallback	Any (no GPU)	—	~5–15	~60–120 sec	Set OLLAMA_NUM_GPU=0; viable for low-volume demos

Docker Compose GPU Configuration

```
# docker-compose.yml – GPU passthrough to Ollama container
services:
  ollama:
    image: ollama/ollama:latest
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
    volumes:
      - ollama_data:/root/.ollama
```

Remove the deploy.resources block entirely for CPU-only operation — no other changes needed.

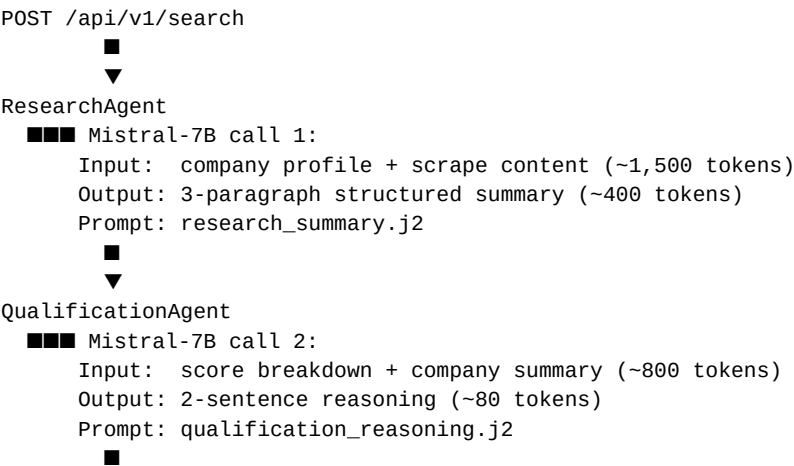
Switching Between CPU and GPU

```
# CPU-only (current)
OLLAMA_NUM_GPU=0

# Single GPU
OLLAMA_NUM_GPU=1

# No change to API, endpoints, or agent code required
# Ollama handles GPU/CPU routing transparently
```

4. LLM Architecture Inside the Pipeline



▼

SalesAgent (qualified leads only)

- nomic-embed-text call:
 - Input: RAG query string (~200 tokens)
 - Output: 768-dim embedding vector → ChromaDB cosine search
- Mistral-7B call 3:
 - Input: 5 RAG chunks + company profile (~3,000 tokens)
 - Output: cold email SUBJECT + BODY (~250 tokens)
 - Prompt: cold_email.j2
- Mistral-7B call 4:
 - Input: pain hook + RAG one-liner + DM name (~600 tokens)
 - Output: LinkedIn message ≤300 chars
 - Prompt: linkedin_message.j2

Total LLM calls per qualified lead: 4 (3× Mistral-7B + 1× nomic-embed-text)

Total LLM calls per disqualified lead: 2 (2× Mistral-7B, SalesAgent skipped)

5. Prompt Templates (Jinja2)

All LLM prompts are managed as version-controlled **Jinja2 templates** in backend/prompts/:

Template	Agent	Purpose
research_summary.j2	ResearchAgent	3-paragraph company analysis (Overview / Pain Points / HRMS Fit Signal)
qualification_reasoning.j2	QualificationAgent	2-sentence reasoning (best signal + biggest risk)
cold_email.j2	SalesAgent	RAG-grounded cold email (150–220 words, SUBJECT/BODY format)
linkedin_message.j2	SalesAgent	LinkedIn outreach (≤300 chars, no emojis)
self_critique.j2	SalesAgent	Hallucination verification (claims vs RAG chunks)

Why Jinja2 templates instead of f-strings:

- Version-controlled alongside code — prompt changes are reviewable in git diff
- Human-readable — non-engineers can review and edit prompts
- Testable — prompts can be rendered and inspected without running the LLM
- Explicit RAG injection — `{% for chunk in rag_chunks %}` makes grounding visible

Summary

Decision	Choice	Rationale
Model	Mistral-7B-Instruct-v0.3	Best balance of quality, context length, license, and VRAM for Indian B2B HRMS use case
Quantization	Q4_K_M	97% quality at 31% VRAM cost — minimum viable quality for structured output tasks
Embeddings	nomic-embed-text (768-dim)	Purpose-built for retrieval, 137M params, negligible VRAM overhead
Inference server	Ollama	Local deployment, CPU/GPU transparent, drop-in model replacement
GPU target	RTX 3070/4060 Ti (8 GB)	Consumer hardware — accessible for development and small-scale production
CPU fallback	OLLAMA_NUM_GPU=0	Demo-viable at 60–120s per lead, zero hardware dependency